IOE LAB-Mini Project Report

ON

Arc Guard: A Servo-Based Directional Intrusion Detection and Radar Visualization System

Submitted to

MUMBAI UNIVERSITY

In partial fulfillment of requirement for the degree of

Bachelor of Engineering

Information Technology Semester-VII

Submitted By

Sr. No	Name	ID No.
1	PRATHAM PAWAR	VU4F2324148
2	PREETI GOUD	VU4F2324145
3	MARIYAM ASHRAFI	VU4S2425018
4	ASHMEET BHAT VERNEKAR	VU4S2425019



Under the guidance of

Dr. Kavita Wagh

Assistant Professor

Department of Information Technology

Vasantdada Patil Pratishthan's College of Engineering and Visual Arts

Sion Mumbai-22

Academic Year 2026-2027

Department of Information Technology
Vasantdada Patil Pratishthan's College of Engineering & Visual Arts
UNIVERSITY OF MUMBAI
(AY 2026-2027)

CERTIFICATE

This is to certify that the project entitled **"Arc Guard: A Servo-Based Directional Intrusion Detection and Radar Visualization System"** is Bonafide work of **PRATHAM PAWAR- VU4F2324148, PREETI GOUD- VU4F2324145, MARIYAM ASHRAFI- VU4S2425018, ASHMEET BHAT VERNEKAR- VU4S2425019**, submitted to University of Mumbai in partial fulfilment of requirement for the award of degree of **"Bachelor of Engineering in Information Technology- Semester-VII"**

Dr. Kavita Wagh
Guide
Assistant Professor

Dr. Pradip Mane
Head of Department (IT)

Dr. Alam N Shaikh
Principal

Project Report Approval for BE.

This project report entitled **"Arc Guard: A Servo-Based Directional Intrusion Detection and Radar Visualization System"** by PRATHAM PAWAR- VU4F2324148, PREETI GOUD- VU4F2324145, MARIYAM ASHRAFI- VU4S2425018, ASHMEET BHAT VERNEKAR- VU4S2425019 is approved for the degree of **Bachelor of Engineering** in **Information Technology**.

Examiners

1. _____

2. _____

Date:

Place: Mumbai – 22

ABSTRACT

Monitoring access to rooms such as laboratories, server rooms and offices is commonly done either with single-point sensors (PIR or a single fixed ultrasonic module), which give poor spatial coverage and many false alarms, or with camera systems, which are costly, computationally heavy and raise privacy concerns. This report presents **Arc Guard**, a camera-free room monitoring system that detects possible movement and environmental anomalies using only distance measurements.

Arc Guard is built from exactly one ESP32-WROOM-32U development board, three ultrasonic ranging modules, one Tower Pro MG90S 180° servo and one MH-FMD buzzer module. Two ultrasonic sensors are fixed and watch separate zones; the third is mounted on the servo and sweeps a configurable 0–180° arc, producing an angle-tagged distance profile of the room. All sensors are powered at 3.3 V, so their echo outputs are safe for the ESP32 inputs without any additional components. The firmware triggers the sensors strictly one at a time, validates and median-filters each reading, learns a per-sensor and per-angle baseline with a robust noise estimate (median absolute deviation), and runs a finite state machine that requires a change to persist before it is reported. Confirmed changes are classified into two separate categories: **movement events** (dynamic changes) and **anomaly events** (objects appearing, disappearing, sudden jumps, unusual patterns and sensor faults).

The ESP32 streams readings and events over Wi-Fi using a WebSocket connection to a Python FastAPI backend that stores history in SQLite and pushes live updates to a web dashboard. The dashboard shows system health, three sensor panels, a radar-style semicircular map built from real scan data, live charts, a filterable event log, and settings that are applied on the device and acknowledged. An Android app shows the same dashboard on a phone and vibrates and raises a notification when an event is reported. The system reports possible movement and anomalies only; it does not identify people.

Keywords: ESP32, ultrasonic ranging, servo scanning, baseline modelling, anomaly detection, WebSocket, FastAPI, radar visualization, Android, privacy-preserving monitoring.

ACKNOWLEDGEMENT

We the students from **Vasantdada Patil Pratishthan's** College of Engineering, Mumbai of Fourth **Year Engineering** in the Department of **'Information Technology'** have great pleasure in presenting our efforts of developing the project named as **"Arc Guard: A Servo-Based Directional Intrusion Detection and Radar Visualization System".**

We would like to express our heartfelt gratitude to **Dr. Pradip Mane, Head of the IT Department,** for providing us with inspiration, invaluable guidance, and timely suggestions throughout our Mini project journey.

We acknowledge our deep sense of gratitude to our Guide, **Dr. Kavita Wagh**, Assistant Professor in Information Technology, who helped us to complete this project work successfully.

INDEX

Chapter Number	Section	Page Number
	Abstract	4
	Acknowledgement	5
	List of Figures and Tables	7
Chapter 1	**Introduction**	**8**
1.1	Introduction	8
1.2	Research Gaps and Problem Statement	8
1.3	Aim and Motivation	8
1.4	Objectives	9
1.5	Scope and Limitations	9
1.6	Summary	9
Chapter 2	**Literature Review**	**10**
Chapter 3	**Methodology**	**12**
3.1	System Architecture	12
3.2	Hardware Components	13
3.3	Software Stack	14
3.4	Ultrasonic Ranging Principle	14
3.5	Measurement Scheduling and Servo Scanning	14
3.6	Detection Algorithm	15
3.7	Communication Flow	17
Chapter 4	**Circuit & Wiring Implementation**	**18**
4.1	Pin Mapping	18
4.2	Breadboard Layout	18
4.3	Power and Ground	19
4.4	GPIO Selection Rationale	20
4.5	Assembly and Demo Setup	20
Chapter 5	**Implementation, Testing and Results**	**22**
5.1	Firmware Implementation	22
5.2	Backend and Database	23
5.3	Web Dashboard	24
5.4	Android Mobile App	24
5.5	Test Plan and Results	26
5.6	Discussion	27
Chapter 6	**Conclusion and Future Scope**	**28**
	Sustainable Development Goals (SDGs)	28
	References	30

LIST OF FIGURES AND TABLES

Figure	Title
3.1	Hardware block diagram
3.2	System architecture
3.3	Sequential measurement schedule
3.4	Detection logic state machine
3.5	Communication flow
4.1	Breadboard wiring diagram
4.2	Demo placement (top view)
5.1	Firmware flow
5.2	Android app: connect screen, overview and room map
5.3	Android app: analytics, event log and settings

Table	Title
3.1	Hardware components
3.2	Software stack
3.3	Default detection parameters
4.1	GPIO pin mapping
4.2	Power connections
5.1	Backend API endpoints
5.2	Database tables
5.3	Hardware test plan
6.1	SDG mapping

CHAPTER 1
INTRODUCTION

1.1 Introduction

As institutions grow, rooms such as server rooms, research laboratories and record rooms need early warning when something changes inside them. Embedded monitoring today tends to fall into two extremes: camera networks that capture far more than is needed, and single standalone sensors that only say “something triggered” without saying where. Arc Guard sits between these extremes. It uses ultrasonic distance measurements, which carry no image of a person, together with a sweeping sensor that adds directional information, and presents the result as a live radar-style map on a web dashboard.

1.2 Research Gaps and Problem Statement

Low-cost monitoring systems usually rely on a fixed Passive Infrared (PIR) sensor or one stationary ultrasonic module. A single fixed sensor covers one narrow cone, gives no direction, and is easily confused by noise because each reading is judged on its own. Camera-based systems solve coverage but need more processing power, more bandwidth and raise privacy concerns for the people who work in the monitored space.

Problem statement: design a camera-free system, using only an ESP32, three ultrasonic sensors, one servo and one buzzer, that (a) covers a wider area than a single sensor, (b) tells the operator in which direction and at what distance a change occurred, (c) separates genuine persistent changes from noise, and (d) shows live and historical results on a dashboard that runs on a local network.

1.3 Aim and Motivation

The aim is to build a low-cost, privacy-preserving monitoring node that learns the normal distance profile of a room, continuously compares new measurements against it, and reports possible movement and environmental anomalies with their direction and distance. Pairing a micro servo with an ultrasonic time-of-flight sensor gives a mechanically scanned range profile — the same idea used in early sonar mapping for mobile robots [1], [2] — at a component cost that a student lab can afford.

1.4 Objectives

- **Three-sensor coverage:** monitor two fixed zones and one scanned arc with three ultrasonic sensors, triggered sequentially so they do not interfere.
- **180° directional scanning:** sweep the third sensor on an MG90S servo with configurable range, step size and speed, tagging every reading with its angle.
- **Baseline modelling:** learn a baseline and a noise level for each fixed sensor and for every scan angle; allow recalibration from the dashboard.
- **Persistent confirmation:** report a change only when it persists for several valid readings, to reduce false alarms.
- **Two event categories:** keep movement events and anomaly events separate.
- **Local alert:** sound the MH-FMD buzzer on confirmed events (configurable).
- **Wireless real-time dashboard:** stream data over Wi-Fi to a backend and a web dashboard usable on a laptop or phone browser, and in an Android app that alerts the user.
- **Privacy preservation:** achieve all of the above without any camera.

1.5 Scope and Limitations

- Ultrasonic sensors measure distance only. The system reports **possible movement** and **anomalies**; it cannot identify people, recognise faces or distinguish people from animals.
- Effective range is roughly 2–400 cm. Soft surfaces (clothing, curtains) absorb sound and angled hard surfaces reflect it away, both of which can produce missed echoes.
- The beam is a cone of about 15°, so objects closer together than the beam width are not separated even if the scan step is smaller.
- The scanning sensor revisits a given angle only once per sweep (about 4 s), so the radar view is a sampled map, not a continuous video.
- The ESP32-WROOM-32U has an antenna socket but no antenna fitted, so Wi-Fi range is short; the access point must be kept close to the board.
- The system runs on a local network; no cloud service is used.

1.6 Summary

Arc Guard combines the privacy of ultrasonic sensing, the directional awareness of a sweeping sensor and the connectivity of the ESP32 into one node. Detection is performed on the ESP32 itself, so the local buzzer works even if the network is down, while the backend and dashboard provide history, visualisation and remote configuration.

CHAPTER 2

LITERATURE REVIEW

The design draws on work in sonar sensing and mapping, background modelling, anomaly detection and IoT communication. The works below were selected because each directly informs a design decision in Arc Guard.

1. **Moravec and Elfes (1985) [1]** built maps of a robot's surroundings from a ring of wide-angle sonar sensors, treating each reading as evidence spread over the beam cone. Arc Guard's scanning sensor follows the same principle — a sequence of angle-tagged ranges builds a spatial profile — and inherits the same limitation that angular resolution is bounded by the beam width.
2. **Elfes (1989) [2]** formalised this into occupancy grids, where each cell holds a probability of being occupied. Arc Guard uses a simpler per-angle baseline instead of a full grid, which fits the ESP32's memory and the 0–180° single-sensor geometry, but the per-angle comparison against learned free space is the same idea.
3. **Borenstein and Koren (1995) [3]** showed that firing multiple ultrasonic sensors close together causes crosstalk — one sensor receiving another's echo — and proposed error-eliminating rapid firing. Arc Guard avoids crosstalk by never having more than one ping in flight and by leaving a gap between pings.
4. **Kleeman and Kuc (2016) [4]** review sonar physics, including specular reflection from smooth angled surfaces and the dependence of the speed of sound on temperature. These effects explain why invalid readings must be expected and filtered rather than treated as detections.
5. **Piccardi (2004) [5]** reviews background subtraction for video, where each pixel is compared with a learned background model. Arc Guard applies the same structure to distances: each fixed sensor and each scan angle has a learned background (baseline) and a per-point noise level.
6. **Chandola, Banerjee and Kumar (2009) [6]** survey anomaly detection and distinguish point anomalies, contextual anomalies and collective anomalies. Arc Guard's SUDDEN_CHANGE, OBJECT_APPEARED/DISAPPEARED and PATTERN categories correspond to point, contextual (persisting in time) and collective (many angles at once) anomalies.
7. **Leys et al. (2013) [7]** argue that outlier thresholds should use the median absolute deviation (MAD) rather than the standard deviation, because the standard deviation is

itself inflated by outliers. Arc Guard's calibration uses the median as the baseline and 1.4826·MAD as the noise estimate, so a few bad echoes during calibration do not distort the threshold.

8. **Al-Fuqaha et al. (2015) [8]** survey IoT protocols (HTTP, MQTT, CoAP, WebSocket and others). For a single device on a local network that must both send data and receive settings, a persistent bidirectional WebSocket [9] was chosen over HTTP polling, and over MQTT, which would require an additional broker service.

Gap analysis. Simple ultrasonic "radar" builds sweep a sensor and draw the raw readings, but do not learn a baseline or confirm changes, so every fluctuation is drawn as a target. Fixed-sensor alarm systems confirm triggers but have no directional coverage. Arc Guard combines sweeping, per-angle robust baselines, persistence-based confirmation, separate movement/anomaly categories and a configurable real-time dashboard, while staying within a fixed, minimal hardware budget.

CHAPTER 3

METHODOLOGY

3.1 System Architecture

Arc Guard is organised in four layers (Fig 3.2). The **sensing layer** contains the three ultrasonic sensors, the servo and the buzzer. The **edge layer** is the ESP32 firmware, which schedules measurements, filters them, maintains baselines, detects events and drives the servo and buzzer. The **backend layer** is a Python FastAPI server on a laptop that receives data over Wi-Fi, stores it in SQLite and serves it to browsers. The **presentation layer** is a browser-based dashboard served by the backend, also available on an Android phone through a companion app (Section 5.4). Detection runs on the ESP32 so that the system keeps working — and the buzzer still sounds — if the network drops.

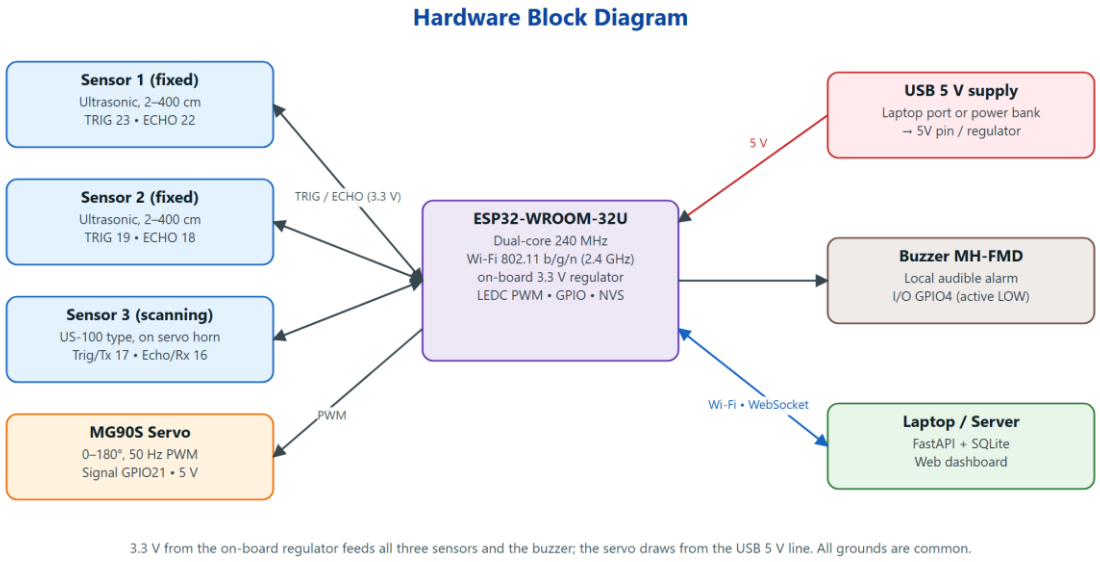


Fig 3.1. Hardware block diagram

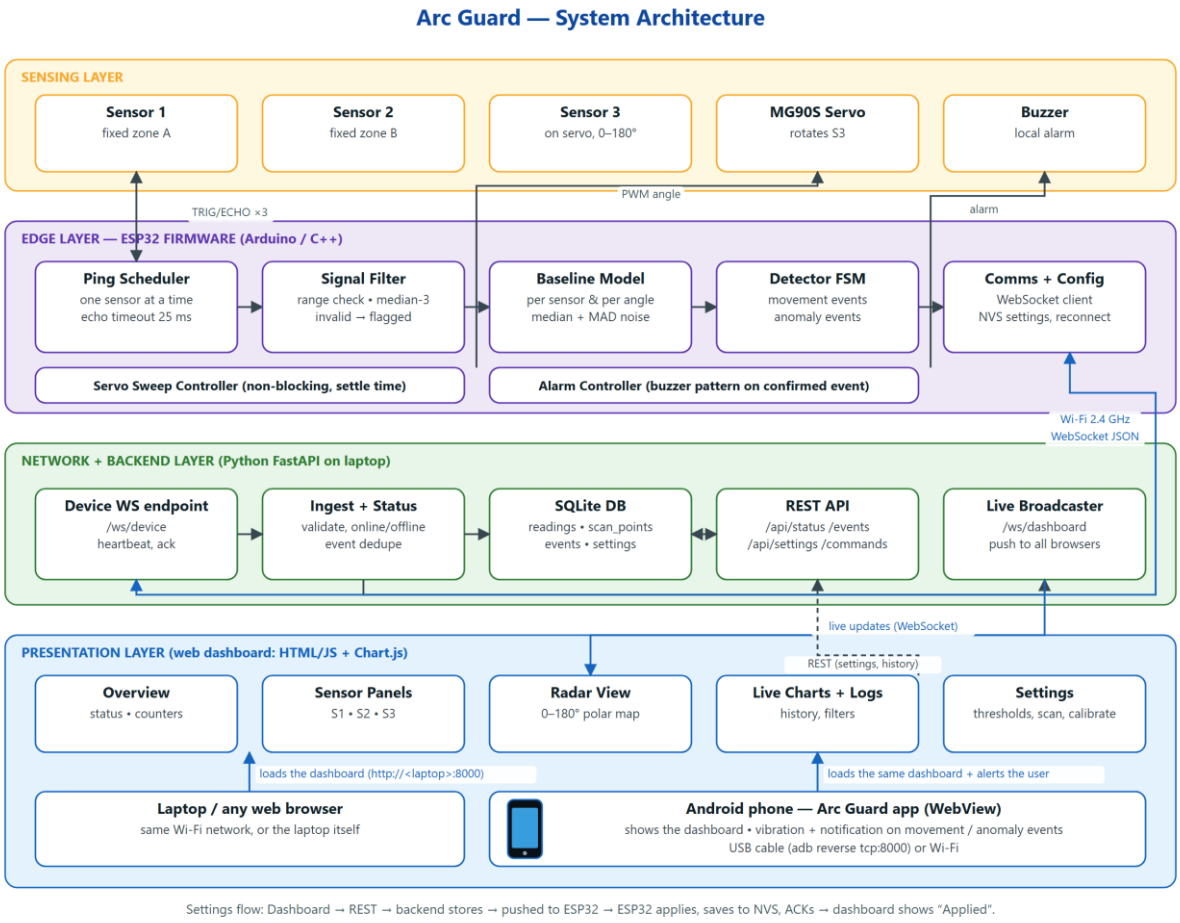


Fig 3.2. System architecture of Arc Guard

3.2 Hardware Components

The hardware is fixed to the components below. No resistors, capacitors or other parts are added; the voltage-level problem of the classic HC-SR04 is avoided by powering the sensors at 3.3 V (Section 4.3).

Table 3.1. Hardware components

Component	Qty	Model / type	Role
Microcontroller	1	ESP32-WROOM-32U on a 38-pin DevKit	Processing, Wi-Fi, PWM, timing
Fixed ultrasonic sensor	2	HC-SR04, 3–5.5 V multi-mode version (2020 board)	Sensor 1 and Sensor 2 zones
Scanning ultrasonic sensor	1	5-pin US-100-type module (VCC, Trig/Tx, Echo/Rx, GND, GND)	Sensor 3 on the servo
Servo motor	1	Tower Pro MG90S, 180°, metal gear	Rotates Sensor 3
Buzzer module	1	MH-FMD, 3-pin (GND, I/O, VCC), low-level trigger	Local audible alarm
Breadboard + jumper wires	—	Full-size breadboard, M-M and F-M jumpers	Interconnection

3.3 Software Stack

Table 3.2. Software stack

Layer	Technology	Reason
Firmware	Arduino framework for ESP32 (C++), PlatformIO or Arduino IDE	Mature libraries; fast to debug; ESP-IDF features not needed at this scale
Firmware libraries	ESP32Servo, ArduinoJson, arduinoWebSockets, Preferences (NVS)	Servo PWM via LEDC, JSON encoding, WebSocket client, persistent settings
Backend	Python 3.11, FastAPI, Uvicorn	Native WebSocket and REST support, async, automatic API docs
Database	SQLite (built-in sqlite3)	Zero configuration, single file, sufficient for one device
Frontend	HTML5, CSS, JavaScript (no build step), Chart.js 4	Served directly by FastAPI, works offline on the LAN; radar drawn on an HTML canvas
Mobile app	Android (Java, WebView), Android 7.0 or later, built with the Android SDK tools	Reuses the web dashboard unchanged; adds phone vibration and notifications
Transport	WebSocket (RFC 6455) over Wi-Fi 2.4 GHz, JSON payloads	Bidirectional, low latency, no extra broker

3.4 Ultrasonic Ranging Principle

Each sensor emits a short burst at 40 kHz when its TRIG input receives a pulse of at least 10 µs, then holds its ECHO output high for the round-trip time t of the sound. The distance is:

$$d = (v \cdot t) / 2, \quad v \approx 331.3 + 0.606 \cdot T \ \ m/s$$

where T is the air temperature in °C. At 25 °C, v ≈ 346.5 m/s, so d (cm) ≈ t (µs) / 57.7. No temperature sensor is used; T is a configurable constant (the speed of sound changes about 0.17 % per °C). A reading is **invalid** if the echo does not arrive within the timeout (25 ms ≈ 4.3 m), or if d is outside 2–400 cm. Invalid readings are flagged and never shown as distances.

The 5-pin US-100-type module also supports a UART mode (selected by a jumper) in which it returns the distance digitally and compensates for temperature internally. The wiring in Chapter 4 works for either mode because the ESP32 can assign its UART to the same two pins.

3.5 Measurement Scheduling and Servo Scanning

Only one sensor is triggered at a time. The scheduler cycles S1 → S2 → S3 with a short gap after each ping so that late echoes die out (Fig 3.3). While S1 and S2 are pinging, the servo moves Sensor 3 to its next angle and settles, so servo movement costs no extra time. With a 5° step, one cycle takes about 110 ms: each fixed sensor is sampled about nine times per

second, and a 0–180° sweep of 37 angles takes about 4 s. The sweep runs 0→180° then 180→0° instead of jumping back, which halves the idle time and avoids a large, current-hungry servo move. Range, step and settle time are configurable from the dashboard.

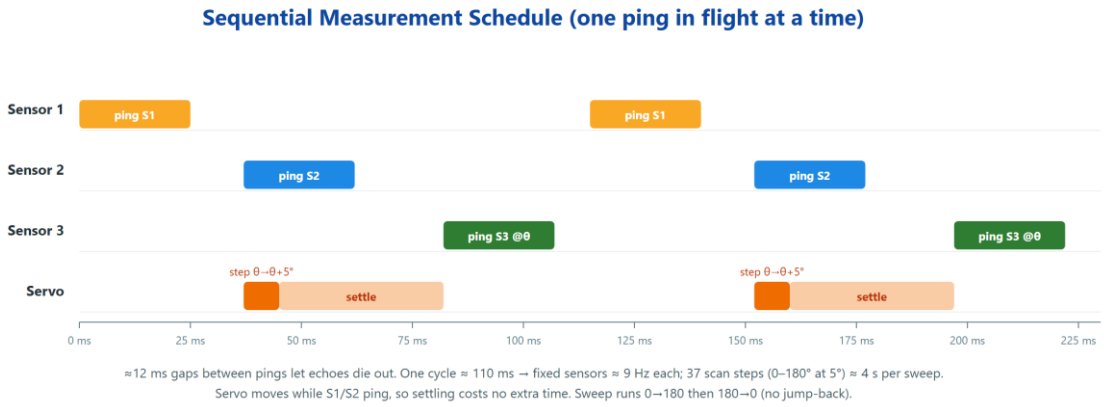


Fig 3.3. Sequential measurement schedule (two cycles shown)

A sensor that keeps timing out is marked FAULT and reported, but the scheduler continues with the other sensors, so one disconnected sensor cannot stop the system.

3.6 Detection Algorithm

A. Baseline calibration

At start-up (and whenever the user presses Calibrate) the firmware collects N = 20 valid readings for each fixed sensor and for each scan angle. The baseline b is their median and the noise level σ is estimated robustly from the median absolute deviation [7]:

$$b = median(x_1 \dots x_N), \qquad \sigma = 1.4826 \cdot median(|x_i - b|)$$

B. Deviation and threshold

Each new valid reading d (median of the last three samples) is compared with the baseline. The threshold adapts to how noisy that particular direction is, but never falls below an absolute minimum:

$$\Delta = d - b, \qquad \tau = max(\tau_abs\ ,\ k \cdot \sigma), \qquad rate\ r = (d_t - d_\{t-1\}) / \Delta t$$

C. Confirmation state machine

Each fixed sensor and each scan angle has its own state machine (Fig 3.4). A reading with $|\Delta| > \tau$ moves the state from IDLE to CANDIDATE. Only after M consecutive valid readings beyond the threshold, lasting at least T_min, is the change CONFIRMED. Invalid readings pause the count instead of resetting it. After an event is emitted the state enters

COOLDOWN so that one person walking past does not produce a stream of duplicate events.

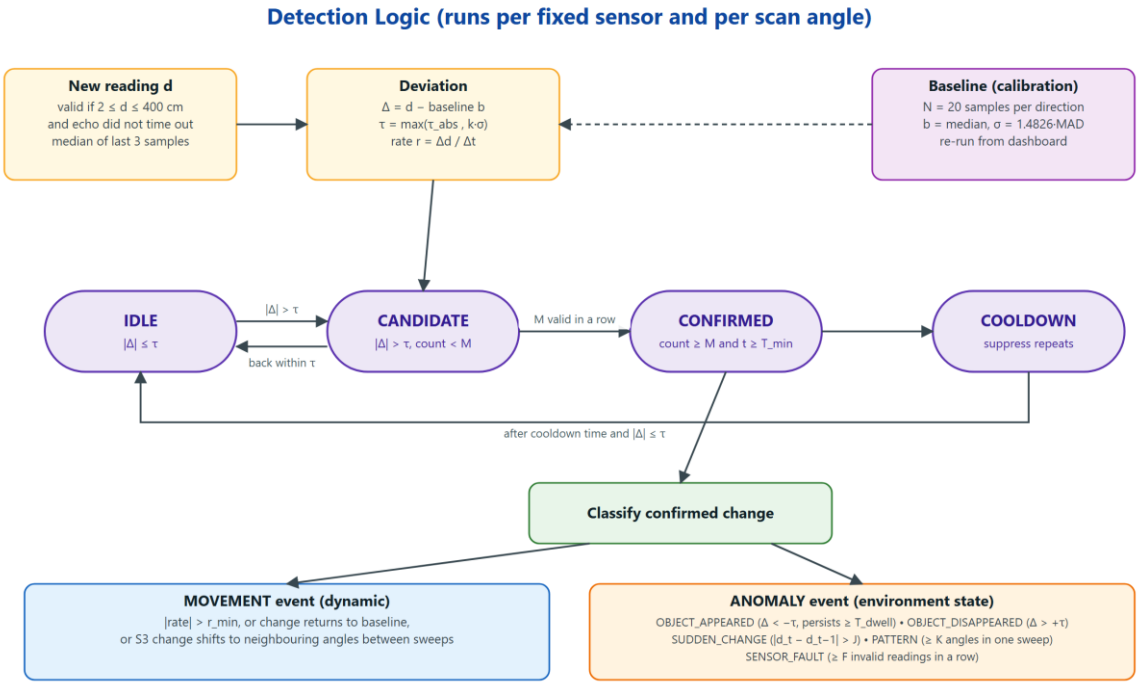


Fig 3.4. Detection logic state machine and event classification

D. Movement versus anomaly

A confirmed change is a **movement event** when it is dynamic: the rate of change exceeds r_min, the reading returns to baseline within the dwell time, or — for the scanning sensor — the changed region shifts to neighbouring angles between sweeps. It is an **anomaly event** when it describes a change in the state of the room: OBJECT_APPEARED (closer than baseline and persisting beyond T_dwell), OBJECT_DISAPPEARED (farther than baseline where a surface used to be), SUDDEN_CHANGE (a jump larger than J between consecutive readings), PATTERN (K or more angles changing within one sweep) and SENSOR_FAULT (F or more invalid readings in a row). The two categories are stored, displayed and filtered separately.

Table 3.3. Default detection parameters (all adjustable from the dashboard)

Parameter	Default	Meaning
τ_abs	15 cm	Minimum deviation treated as a change
k	4	Multiplier on the measured noise σ
M	3	Consecutive valid readings to confirm (fixed sensors); 2 sweeps for S3
T_min	300 ms	Minimum duration of a change
r_min	30 cm/s	Rate above which a change is called movement
T_dwell	10 s	Persistence before OBJECT_APPEARED / DISAPPEARED
J	80 cm	Jump size for SUDDEN_CHANGE

K	15 angles	Newly changed angles in one sweep for PATTERN (~40 % of the arc)
F	10	Invalid readings in a row for SENSOR_FAULT
Cooldown	3 s	Suppression time after an event
Scan range / step / settle	0–180° / 5° / 40 ms	Servo sweep settings

These defaults are starting values; they will be tuned against the false-positive rate measured during testing (Section 5.5). A rule-based detector was chosen over machine learning because it is transparent, needs no training data and can be tuned live; a learned model is listed as future work.

3.7 Communication Flow

The ESP32 connects to the backend's /ws/device WebSocket and sends a hello message with its firmware version and current settings. It then sends a telemetry batch every 200 ms, events immediately when they are confirmed, and a heartbeat every 2 s. The backend stores readings and events in SQLite and broadcasts them to all open dashboards on /ws/dashboard. When a user changes a setting, the dashboard calls the REST API; the backend stores the value and forwards it to the ESP32, which applies it, saves it in non-volatile storage (NVS) and replies with an acknowledgement. Only then does the dashboard show the setting as Applied — if the device is offline the setting stays Pending. If no heartbeat arrives for 6 s the device is shown as Offline; the firmware reconnects automatically when Wi-Fi returns (Fig 3.5).

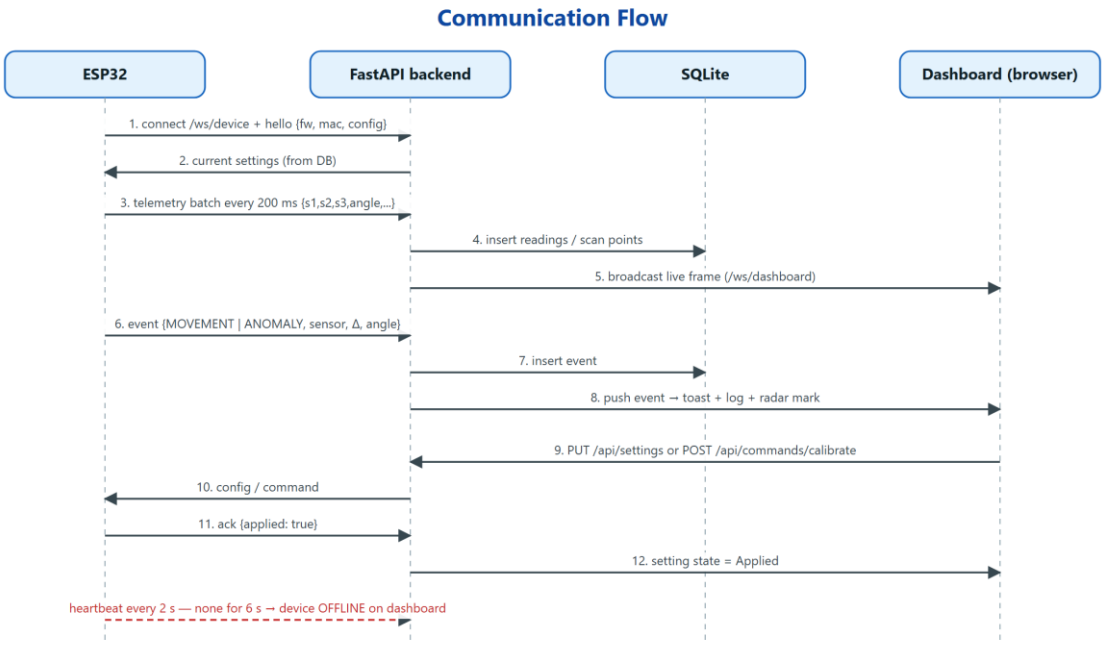


Fig 3.5. Communication flow between ESP32, backend, database and dashboard

CHAPTER 4

CIRCUIT & WIRING IMPLEMENTATION

4.1 Pin Mapping

All signal pins are on the right-hand header of the 38-pin DevKit, which is the header that plugs into the breadboard. Every pin used is a general-purpose pin with no boot-strapping or flash function.

Table 4.1. GPIO pin mapping

Function	ESP32 GPIO	Breadboard row (col. a)	Wire colour	Direction
Sensor 1 TRIG	GPIO23	7	Yellow	Output
Sensor 1 ECHO	GPIO22	8	Green	Input (3.3 V)
Servo signal	GPIO21	11	Orange	PWM output, 50 Hz
Sensor 2 TRIG	GPIO19	13	Blue	Output
Sensor 2 ECHO	GPIO18	14	Purple	Input (3.3 V)
Sensor 3 Trig/Tx	GPIO17	16	Pink	Output (or UART RX in UART mode)
Sensor 3 Echo/Rx	GPIO16	17	Teal	Input (or UART TX in UART mode)
Buzzer I/O	GPIO4	18	Brown	Output, LOW = sound
Sensors + buzzer VCC	3V3 pin	Right + rail	Red	Power 3.3 V
Servo VCC	5V pin	Left + rail	Dark red	Power 5 V
All grounds	GND pins	Both − rails (bridged)	Black	Common ground

4.2 Breadboard Layout

The 38-pin DevKit is wider than the usable area of a standard breadboard, so its right-hand header is inserted in column **a** (rows 6–24) and the left-hand header hangs off the edge. Each ESP32 pin then has four free holes (columns b–e) in its row for connecting wires. The 3V3, 5V and GND pins on the left header are reached with female-to-male jumpers. Fig 4.1 shows the complete layout.

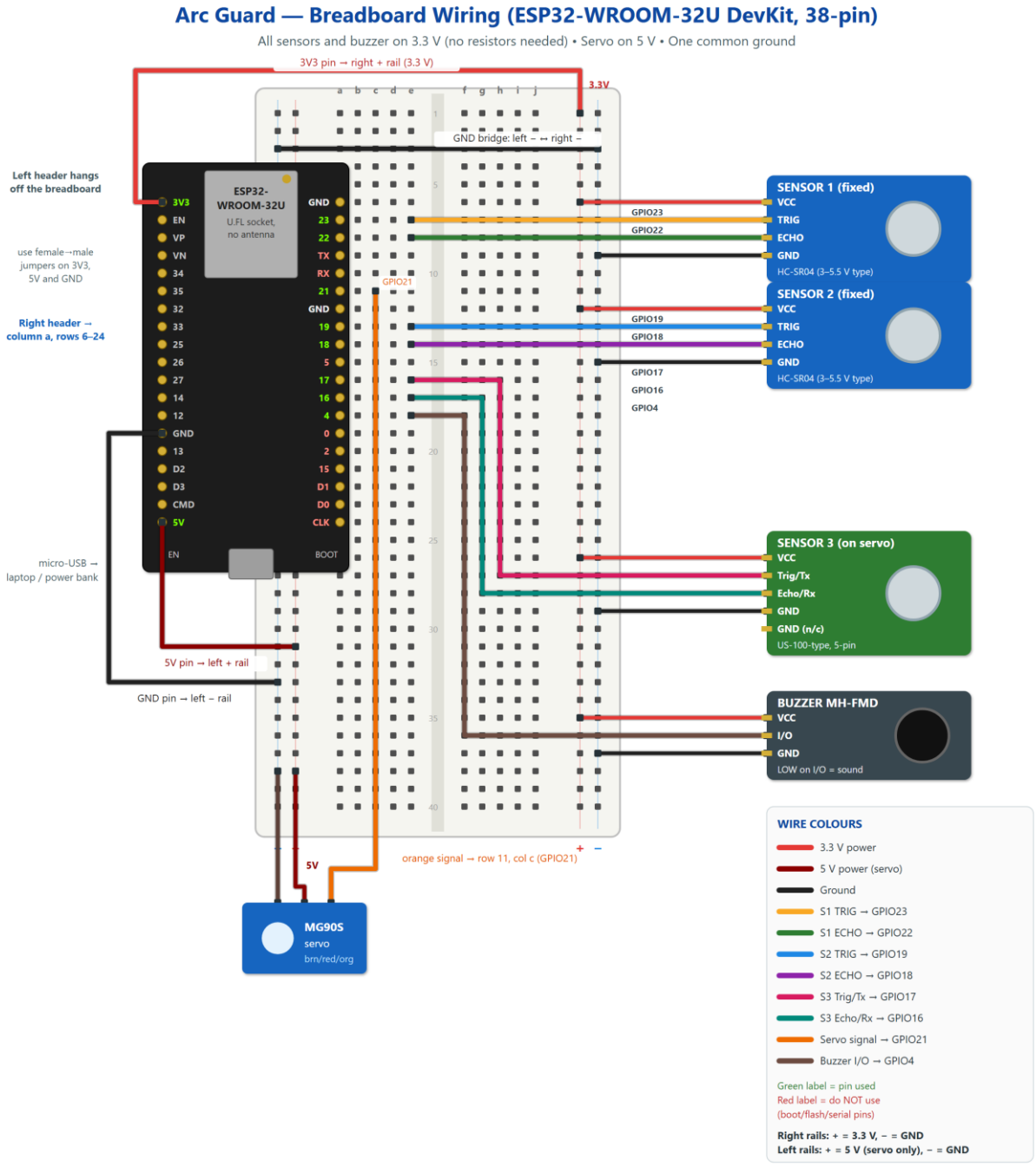


Fig 4.1. Breadboard wiring diagram (all connections)

4.3 Power and Ground

The classic HC-SR04 is a 5 V device whose ECHO output swings to 5 V, while ESP32 inputs are rated only to 3.6 V; connecting them directly would risk damaging the chip, and a resistor divider would add components. The sensors used in this project are newer versions rated for 3–5.5 V (the two HC-SR04 boards) and 2.4–5.5 V (the US-100-type module). Powering them from the ESP32's 3.3 V regulator means their ECHO outputs can never exceed 3.3 V, which is safe for the ESP32 with no extra parts. The 3.3 V TRIG signal from

the ESP32 is sufficient to trigger them. The buzzer is also powered at 3.3 V so that a HIGH level on its I/O pin fully switches it off.

Table 4.2. Power connections

Rail / node	Source	Loads	Approx. current
Right + rail (3.3 V)	ESP32 3V3 pin (on-board regulator)	3 sensors, buzzer	≈ 3 × 15 mA + 30 mA
Left + rail (5 V)	ESP32 5V pin (USB 5 V)	MG90S servo only	≈ 150–250 mA moving; up to ≈ 700 mA stalled
Both − rails	ESP32 GND pins (bridged)	Everything	Common return

The MG90S draws current spikes when it moves. Together with Wi-Fi transmit peaks of a few hundred milliamperes, these can cause a brown-out reset if the USB port is weak. Because no extra capacitor or supply is allowed, this is handled in software and in setup: small angle steps, no large jump-back move, detaching the servo signal when scanning is stopped, and powering the board from a good USB port or a power bank. The firmware reports the reset reason, so brown-outs are visible on the dashboard.

4.4 GPIO Selection Rationale

- **Avoided — GPIO 6–11 (CLK, D0–D3, CMD):** connected to the on-module SPI flash; using them crashes the board.
- **Avoided — GPIO 0, 2, 5, 15:** boot-strapping pins whose level at reset selects the boot mode or boot behaviour; a sensor holding them low can stop the board from booting or uploading. GPIO2 also drives the on-board LED on most DevKits.
- **Avoided — GPIO 1 and 3 (TX/RX):** used by the USB serial port for uploading and the Serial Monitor.
- **Used — GPIO 4, 16, 17, 18, 19, 21, 22, 23:** general-purpose pins on the same header with no special boot function. GPIO16/17 are free on WROOM-32/32U modules (they are only reserved on WROVER modules with PSRAM).
- Signal pairs were chosen on adjacent rows (23/22, 19/18, 17/16) so each sensor's TRIG and ECHO wires run side by side.

4.5 Assembly and Demo Setup

1. Insert the ESP32 right-hand header into column a, rows 6–24, with the antenna end towards row 1. The left header hangs off the board.
2. Connect 3V3 (left header, top) to the right + rail; this rail is now 3.3 V. Connect 5V (left header, bottom) to the left + rail; this rail is 5 V and is used **only** for the servo.

3. Connect a left-header GND to the left − rail, and bridge the left − rail to the right − rail at row 3.
4. Sensor 1: VCC → right + rail, GND → right − rail, TRIG → row 7 (GPIO23), ECHO → row 8 (GPIO22).
5. Sensor 2: VCC → right + rail, GND → right − rail, TRIG → row 13 (GPIO19), ECHO → row 14 (GPIO18).
6. Sensor 3 (on servo): VCC → right + rail, one GND → right − rail, Trig/Tx → row 16 (GPIO17), Echo/Rx → row 17 (GPIO16). Use long, loose jumpers so the wires do not pull on the sensor as it rotates.
7. Servo: brown → left − rail, red → left + rail (5 V), orange → row 11 (GPIO21).
8. Buzzer: VCC → right + rail (3.3 V), GND → right − rail, I/O → row 18 (GPIO4).
9. Before powering, check with a multimeter that the right + rail is not shorted to the 5 V rail and that no sensor VCC goes to 5 V.
10. Place the unit as in Fig 4.2: the scanning arc faces the demo area, Sensors 1 and 2 face outside the arc, and the phone hotspot is within 1–2 m of the ESP32.

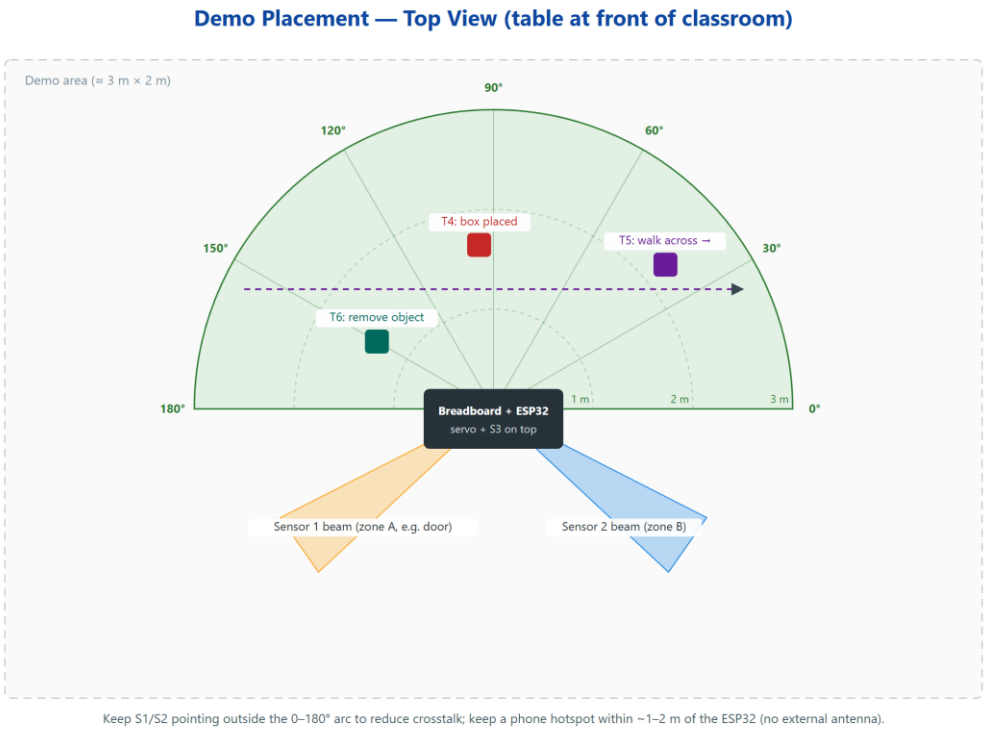


Fig 4.2. Demo placement (top view) with test positions T4–T6

CHAPTER 5

IMPLEMENTATION, TESTING AND RESULTS

Development follows six phases: hardware verification, individual hardware tests, firmware, backend, frontend and full integration. Each phase is tested before the next begins. Any simulated data used during development is marked as simulated in the database and shown with a SIMULATED banner on the dashboard, so it can never be mistaken for real measurements.

Two simulators support development. A scripted simulator replays a fixed scenario, and a **3D room simulator** (built with Three.js and served at /sim) models a virtual room in which boxes and a person can be moved; each virtual sensor measures distance by casting a cone of rays matching the ~15° ultrasonic beam, and a JavaScript port of the firmware's detection code produces events from those distances. Running it exposed two flaws that were then fixed in the firmware as well: one object spanning several scan angles produced one event per angle (events are now grouped per object), and the PATTERN threshold was too low (raised to 15 angles). Simulator results verify the software pipeline only; they are not hardware results.

5.1 Firmware Implementation

The firmware is written in C++ on the Arduino framework and split into modules: configuration (NVS), ultrasonic driver, scheduler, servo sweep controller, baseline model, detector, alarm controller and network client. The main loop never blocks for long: pings are timed with micros(), all other work is scheduled with millis(), and the WebSocket client and Wi-Fi reconnection run alongside measurement. The Serial Monitor prints each reading, state change and event for debugging.

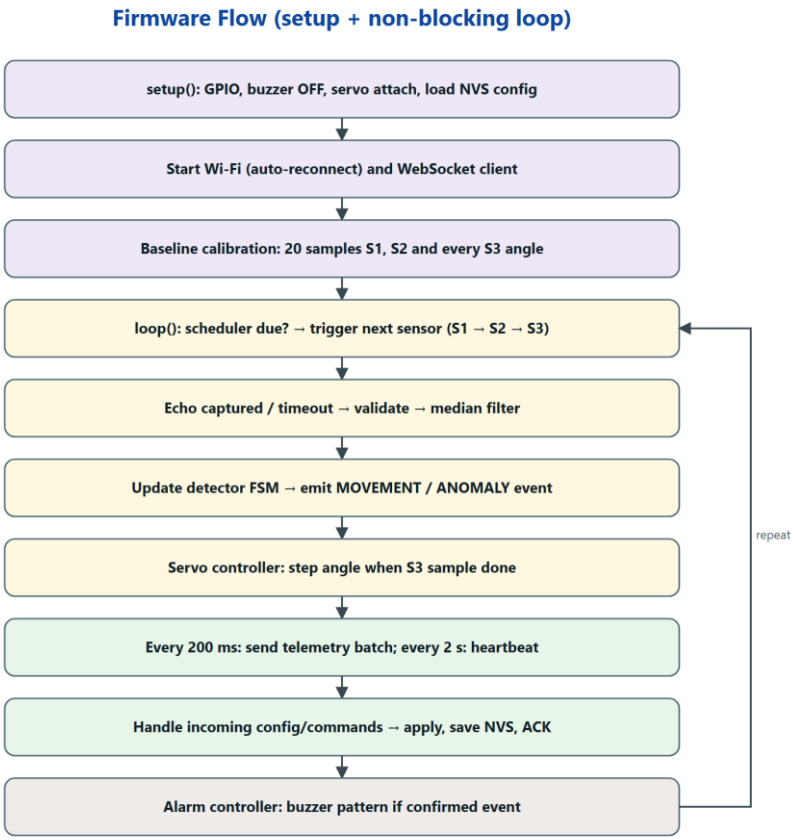


Fig 5.1. Firmware flow

5.2 Backend and Database

The backend is a FastAPI application served by Uvicorn on the laptop. It accepts one device connection and any number of dashboard connections, stores data in SQLite, tracks whether the device is online, and exposes the REST endpoints below.

Table 5.1. Backend API endpoints

Endpoint	Method	Purpose
/ws/device	WebSocket	ESP32 telemetry, events, heartbeats; receives settings and commands
/ws/dashboard	WebSocket	Live frames and events pushed to browsers
/api/status	GET	Device online state, Wi-Fi RSSI, uptime, counters, last update
/api/readings	GET	Historical readings filtered by sensor and time range
/api/scan/latest	GET	Most recent complete sweep (angle, distance, baseline)
/api/events	GET	Event log filtered by sensor, category, date and status
/api/settings	GET / PUT	Read and change detection and scan settings
/api/events/{id}/ack	POST	Mark an event as acknowledged
/api/events.csv	GET	Export the filtered event log as CSV
/api/events/histogram	GET	Movement / anomaly counts per time bucket

/api/commands/{calibrate | scan_start | scan_stop | buzzer_test | silence}	POST	Device commands, acknowledged by the ESP32

Table 5.2. Database tables

Table	Main columns
readings	id, ts, sensor_id, distance_cm, valid, angle_deg, source (real / simulated)
scan_points	id, sweep_id, ts, angle_deg, distance_cm, baseline_cm, valid
events	id, ts, sensor_id, category (MOVEMENT / ANOMALY), type, distance_cm, baseline_cm, delta_cm, angle_deg, status, source
settings	key, value, updated_at, applied_at
device_status	ts, online, rssi, uptime_s, free_heap, reset_reason

5.3 Web Dashboard

The dashboard is a single-page web application (plain HTML, CSS and JavaScript with Chart.js, served by the backend and needing no build tools or internet access) with six sections: an **overview** (system, ESP32 and Wi-Fi status, active sensors, event counters, last activity, last update); three **sensor panels** (current, previous and baseline distance, deviation, status; for Sensor 3 also angle, scan progress and last completed scan); a top-view **room map** that draws Sensor 3's 0–180° scan (baseline contour, latest sweep, servo angle, changed angles) at the front and the beams of the fixed Sensors 1 and 2 at their configured mounting directions, each with its baseline, ±threshold band, live echo and a short trail of recent echoes; a side-view **beam view** for each fixed sensor; a 3D **live room view** in which the user models the room, drags the fixed sensors to their real positions and verifies them against the real readings (auto-aim picks the direction whose predicted distance matches the sensor's calibrated baseline, and Sensor 3's real scan outlines the walls); a prompt asking whether a fixed sensor was moved when its view stays changed without movement for 30 s (ultrasound alone cannot tell a moved sensor from an object left in front of it); **live charts** of distance versus time for each sensor, deviation versus time, event history and distance versus angle; a filterable **event log**; and **settings** for enabling sensors, sensitivity, thresholds, calibration, scan range, resolution, speed and start/stop. Every control round-trips to the ESP32 and shows Pending until it is acknowledged.

[Insert dashboard screenshots here after integration testing — overview, radar view, event log, settings.]

5.4 Android Mobile App

To let the room be watched from a phone, a small Android app (package com.arcguard.app, Android 7.0 or later) was written in Java. Rather than re-implementing the dashboard, the app displays the backend's web dashboard full-screen in an Android WebView [18], so the

phone always shows the same live data, map, charts, event log and settings as the laptop, and any dashboard change appears in the app without rebuilding it. The app adds the parts a browser tab cannot provide well:

- **Event alerts:** the dashboard page passes each new event to the app through a JavaScript interface; the app vibrates (a longer pattern for movement than for an anomaly) and, when it is in the background, posts an Android notification that opens the Events tab when tapped.
- **Connect screen:** the backend address is entered once and stored; a Test button checks /api/status before connecting. Options switch the alerts on or off and keep the screen awake. It can be reopened from an “App” button that the dashboard shows only inside the app.
- **Connection handling:** if the backend cannot be reached, a “Not connected” screen is shown and the app retries every 5 s; CSV exports from the event log are saved to the phone's Downloads folder.

The phone reaches the backend in one of two ways. Over **Wi-Fi**, it uses the laptop's local IP address (http://<laptop-ip>:8000). Over a **USB cable**, the command adb reverse tcp:8000 tcp:8000 forwards the phone's port 8000 to the laptop, so the app uses http://localhost:8000; this works even without a shared Wi-Fi network. The app is built and installed by a script that calls the Android SDK tools directly (aapt2, javac, d8, apksigner), so no Gradle setup or internet access is needed. Figs 5.2 and 5.3 are screenshots of the app on an Android 13 phone connected over USB. Because the hardware had not yet been tested when they were taken, the backend was fed by the project's device simulator (simulator.py), which connects exactly like the ESP32; the dashboard therefore shows its SIMULATED DATA banner and SIM tags. The screenshots show the user interface working end to end; the readings and events in them are not hardware results.

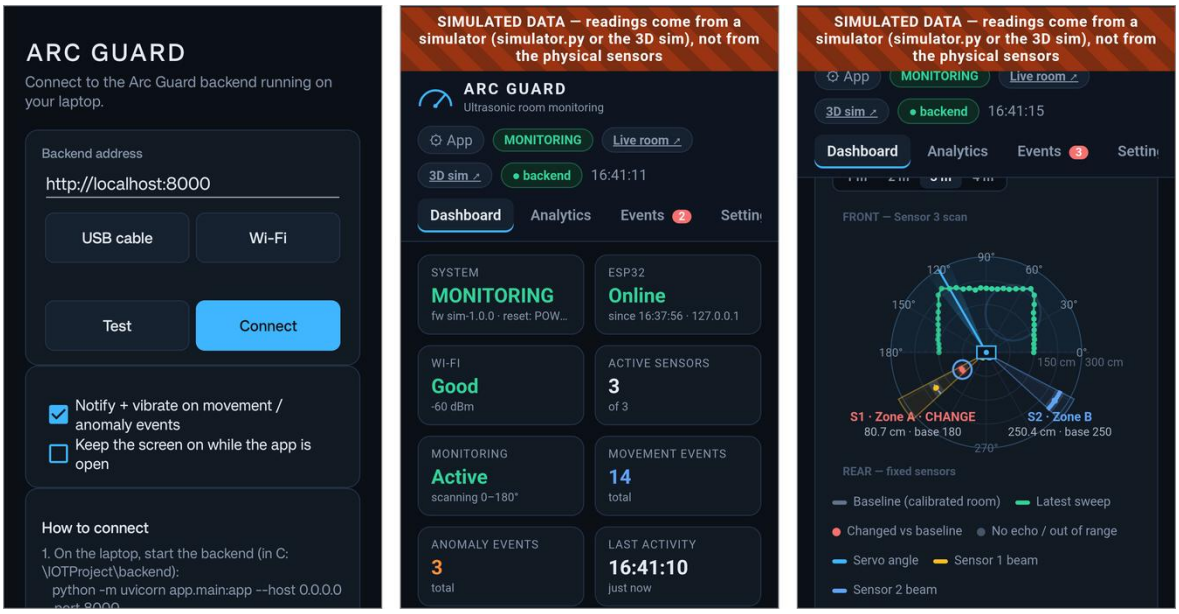


Fig 5.2. Android app: connect screen, dashboard overview and room map (simulated data)

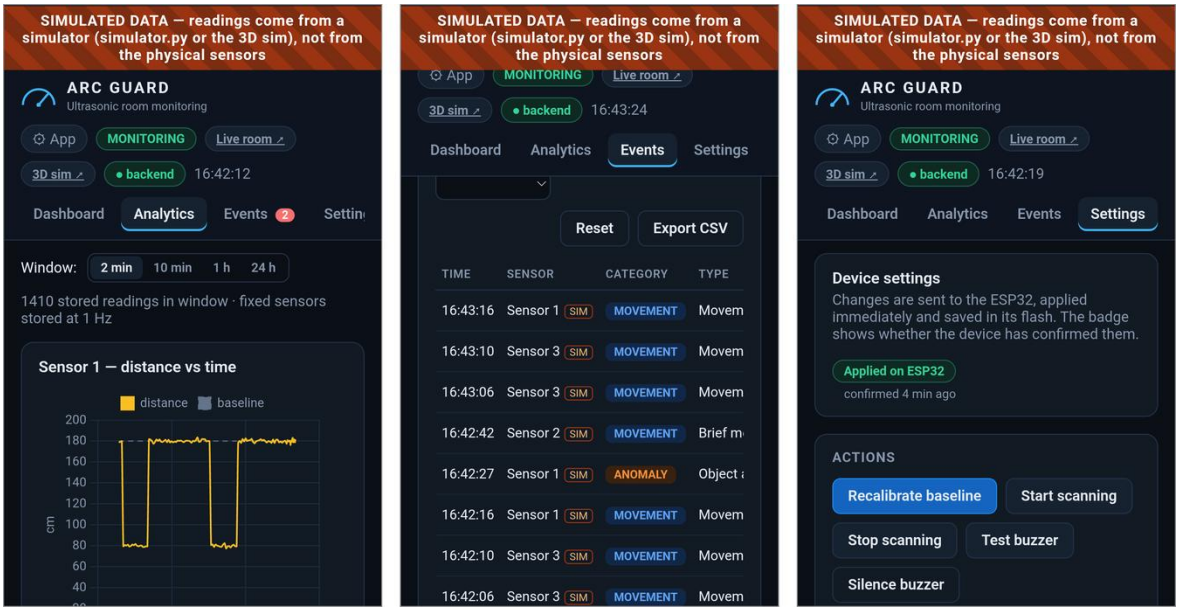


Fig 5.3. Android app: analytics, event log and device settings (simulated data)

Alerts depend on the app staying open: Android may stop an app that has been in the background for a long time, so the phone is a convenience display and the buzzer on the ESP32 remains the primary local alarm.

5.5 Test Plan and Results

The following tests will be run on the physical hardware. The **Observed** column is to be completed during testing with measured values and screenshots; it is intentionally left blank here so that no result is reported before it has been measured.

Table 5.3. Hardware test plan

#	Test	Procedure	Expected dashboard result	Observed
1	Normal room	Calibrate, leave the area empty for 5 min	All sensors OK, no events, radar matches room outline	
2	Object in S1 zone	Place a box ~1 m in front of Sensor 1	S1 deviation shown; MOVEMENT on entry; OBJECT_APPEARED after T_dwell	
3	Object in S2 zone	Same as test 2 for Sensor 2	Same as test 2 for S2 only	
4	Object in scan area	Place a box at about 90°, 1.5 m (T4)	Changed angles near 90° marked on radar; OBJECT_APPEARED with angle	
5	Movement across scan	Walk slowly across the arc (T5)	Changed region moves across angles; MOVEMENT events with changing angle	
6	Object removed	Calibrate with a box present, then remove it (T6)	OBJECT_DISAPPEARED at that angle	
7	Sensor fault	Unplug Sensor 2 ECHO wire	S2 status FAULT, SENSOR_FAULT event; S1 and S3 keep working	
8	Wi-Fi loss	Turn the hotspot off for 30 s, then on	Device Offline after ~6 s; buzzer still works; reconnects automatically	
9	Phone alert	Android app in the background; walk across a sensor zone	Phone vibrates and shows a movement notification; tapping it opens the Events tab	

For each test, false positives will be counted over a fixed period of an empty room, and detection delay will be measured from the moment the object is placed to the event appearing on the dashboard.

5.6 Discussion

The design choices target the main weaknesses identified in Chapter 2. Sequential triggering removes crosstalk between the three sensors. Robust per-angle baselines let noisy directions (corners, angled furniture) have higher thresholds without making quiet directions less sensitive. Persistence-based confirmation and the cooldown state suppress single bad echoes and repeated alerts. Separating movement from anomaly events makes the log easier to interpret: a person walking past and a bag left in a corner are different situations and are reported differently.

The main limitations are physical. The 15° beam limits angular resolution; soft or angled targets may be missed; the scanning sensor samples each angle only once per sweep; and without an external antenna the Wi-Fi range is short. These limitations are stated on the dashboard and in this report rather than hidden.

CHAPTER 6

CONCLUSION AND FUTURE SCOPE

Arc Guard shows that useful directional monitoring of a room can be built from one ESP32, three ultrasonic sensors, one servo and one buzzer, with no camera and no additional electronic components. Careful pin selection and 3.3 V sensor power make the circuit safe; sequential scheduling and a sweeping sensor provide coverage and direction; robust baselines and a confirmation state machine aim to keep false alarms low; and a WebSocket-based backend and dashboard give live and historical visibility with settings that genuinely reach the device. The final evaluation of detection accuracy and false-positive rate will be recorded from the tests in Section 5.5.

Possible enhancements, which do not change the core design:

- **Occupancy-grid mapping:** accumulate sweeps into a 2-D grid [2] for a richer room map.
- **Adaptive baselines:** slowly update the baseline during long quiet periods, with safeguards so that a real anomaly is not absorbed.
- **Focus scanning:** after a change, rescan a narrow sector more densely before resuming the full sweep.
- **Learned classifier:** once enough labelled event data has been recorded, compare a small TinyML model against the rule-based detector and adopt it only if it measurably improves accuracy.
- **Multi-node deployment:** several Arc Guard nodes reporting to one backend to cover larger floors.
- **Push notifications:** alerts sent from the backend (for example through Firebase Cloud Messaging) so the phone is alerted even when the Android app is closed, plus optional e-mail or messaging alerts.

Sustainable Development Goals (SDGs)

The project relates to the United Nations Sustainable Development Goals [17] as follows:

Table 6.1. SDG mapping

SDG	Contribution
SDG 9 — Industry, Innovation and Infrastructure	A low-cost, open, embedded approach to monitoring built from common components.
SDG 11 — Sustainable Cities and Communities	Improves safety of indoor spaces while respecting the privacy of occupants.
SDG 12 — Responsible	Achieves the goal with a minimal component count and a

Consumption and Production	low-power microcontroller.
SDG 16 — Peace, Justice and Strong Institutions	Supports protection of controlled-access institutional rooms without surveillance cameras.

REFERENCES

[1] H. Moravec and A. Elfes, “High resolution maps from wide angle sonar,” in Proc. IEEE Int. Conf. Robotics and Automation (ICRA), St. Louis, MO, USA, 1985, pp. 116–121.

[2] A. Elfes, “Using occupancy grids for mobile robot perception and navigation,” Computer, vol. 22, no. 6, pp. 46–57, Jun. 1989.

[3] J. Borenstein and Y. Koren, “Error eliminating rapid ultrasonic firing for mobile robot obstacle avoidance,” IEEE Trans. Robotics and Automation, vol. 11, no. 1, pp. 132–138, Feb. 1995.

[4] L. Kleeman and R. Kuc, “Sonar sensing,” in Springer Handbook of Robotics, B. Siciliano and O. Khatib, Eds., 2nd ed. Cham, Switzerland: Springer, 2016, pp. 753–782.

[5] M. Piccardi, “Background subtraction techniques: a review,” in Proc. IEEE Int. Conf. Systems, Man and Cybernetics, The Hague, Netherlands, 2004, vol. 4, pp. 3099–3104.

[6] V. Chandola, A. Banerjee, and V. Kumar, “Anomaly detection: A survey,” ACM Computing Surveys, vol. 41, no. 3, Art. 15, Jul. 2009.

[7] C. Leys, C. Ley, O. Klein, P. Bernard, and L. Licata, “Detecting outliers: Do not use standard deviation around the mean, use absolute deviation around the median,” J. Experimental Social Psychology, vol. 49, no. 4, pp. 764–766, 2013.

[8] A. Al-Fuqaha, M. Guizani, M. Mohammadi, M. Aledhari, and M. Ayyash, “Internet of Things: A survey on enabling technologies, protocols, and applications,” IEEE Communications Surveys & Tutorials, vol. 17, no. 4, pp. 2347–2376, 2015.

[9] I. Fette and A. Melnikov, “The WebSocket Protocol,” IETF RFC 6455, Dec. 2011.

[10] Espressif Systems, “ESP32 Series Datasheet.” [Online]. Available: https://www.espressif.com/en/support/documents/technical-documents

[11] Espressif Systems, “ESP32-WROOM-32D & ESP32-WROOM-32U Datasheet.” [Online]. Available: https://www.espressif.com/en/support/documents/technical-documents

[12] ElecFreaks, “Ultrasonic Ranging Module HC-SR04” (datasheet).

[13] Tower Pro, “MG90S Metal Gear Micro Servo” (datasheet).

[14] Espressif Systems, “Arduino core for the ESP32.” [Online]. Available: https://github.com/espressif/arduino-esp32

[15] S. Ramírez, “FastAPI documentation.” [Online]. Available: https://fastapi.tiangolo.com

[16] SQLite Consortium, “SQLite documentation.” [Online]. Available: https://www.sqlite.org/docs.html

[17] United Nations, “Transforming our world: the 2030 Agenda for Sustainable Development,” A/RES/70/1, 2015.

[18] Google, “WebView,” Android Developers documentation. [Online]. Available: https://developer.android.com/reference/android/webkit/WebView